

电子科技大学

专业学位研究生学位论文中期考评表

攻读学位级别： ☐ 博士 ☒ 硕士

培养方式： ☒ 全日制 ☐ 非全日制

专业学位类别或领域： 计算机技术

学 院： 计算机科学与工程学院

学 号： 202422081113

姓 名： 王 威

论 文 题 目： 面向非连续时间约束的区块链
数据共享关键技术研究及实现

校内指导教师： 高建彬

校外指导教师： 王鹏

填 表 日 期： 2026 年 7 月 27 日

电子科技大学研究生院

一、已完成的主要工作

1. 开题报告通过时间：2025 年 12 月 24 日（以开题记录为准）	
2. 课程学习情况	
是否已达到培养方案规定的学分要求	☒ 是 ☐ 否
3. 论文研究进展	
按照开题计划，填写开题以来学位论文工作的研究进展。视具体研究内容，可包括理论、计算、实验（或实证）等方面（可续页）	
（1）研究背景与问题动因 随着医疗健康、政务协同、工业互联网和供应链金融等场景持续数字化，数据逐渐由单一机构内部资源转变为跨主体协同要素，跨组织数据共享正在成为数字经济运行的基础环节。这类场景普遍存在两个基本特征：一是数据的产生、存储与使用主体相分离，数据所有者希望在不泄露原始数据的前提下向授权用户提供使用价值；二是共享过程受到法规与业务约束，访问必须被限制在明确的主体范围、用途范围与时间范围之内。由此形成数据所有权与使用权分离下的核心矛盾：一方面要保证数据可用，另一方面要保证访问可控、过程可审计、失效可追溯。仅依靠传统数据库的访问控制列表或应用层权限校验^{[1][2][3][4]}，难以满足跨组织场景下多方共同维护授权事实、事后追溯责任边界的需求。 区块链凭借不可篡改账本、智能合约和可追溯事件记录，为多方环境下的状态一致性与责任审计提供了天然的技术基础^{[5][6][7]}：多个互不信任的组织可以围绕同一份公开账本达成对授权状态的一致认知^{[5][6]}，任何状态变化都可以被审计和复核。但区块链本身并不提供数据机密性，将业务数据直接写入链上会造成隐私暴露和高额存储成本，也不符合数据所有者对数据控制权的要求。因此，把密文数据保存在链下、把授权状态与审计记录维护在链上，成为区块链数据共享的基本架构选择。然而这一架构引出一个关键问题：链上维护的授权记录只有在真正约束链下密钥与密文材料释放时才有意义，否则授权记录只是事后审计的附件，无法在访问发生前阻止未授权使用。 从时间维度看，面向数据共享的授权策略往往不是单一连续区间。在排班访问、周期授权、跨日窗口和节假日例外等业务中，时间条件通常由多个不连续、非对齐的窗口组成，并可能伴随例外日期、周期性片段和临时开放窗口。例如，一项授权可能仅在工作日的若干时段有效，也可能由多个跨日窗口与例外日期共同构成。传统的连续时间模型难以表达这类由多个片段拼合而成的授权规则；而如果直接把每个最小时间单元写入策略，策略的表示规模、密文头部大小和策略处理成本又会随时间域范围线性膨胀，难以支撑长时间、细粒度的授权场景。 非连续时间约束还给策略表示本身带来了更深层的问题：同一组授权规则可以由不同输入序列表达，这些序列可能因乱序、重复、相交、相邻、嵌套或等价拆分而互不相同，却对应相同的允许时间集合。如果这些表示直接参与摘要计算或授权判断，会产生不同的字节表示和不同的策略标识，进而破坏跨组件校验的一致性——策略编译组件、授权签发组件、验	

证组件与审计组件各自计算出的策略摘要可能互不一致。因此，非连续时间约束首先要求回答“如何获得确定、唯一、可复现的语义表示与策略摘要”这一基础问题。

然而，仅有策略表示并不足以构成可信授权。在动态授权环境中，用户即使持有满足策略的长期凭证，其访问仍可能因暂停、撤销、策略更新或密钥轮换而不再有效；授权状态的变化必须被所有参与方一致地感知和执行。若仅依赖本地或单一服务端校验，则难以应对状态变更、跨实例重放以及能力在链与合约上下文之间迁移等场景：本地保存的策略副本可能过期，单一验证服务可能被绕过程序直接访问资源，同一能力可能在多个验证实例上被重复消费。区块链能够提供确定性的公开状态与可审计的状态变迁，但如何把策略摘要、资源状态、用户密钥版本与授权时效锚定到链上，并使授权能力只能用于指定链、指定合约实例与指定资源状态，是需要专门设计的第二个问题。

在链上授权状态确定之后，还存在链下密文对象生命周期与链上状态之间的失配问题。链上记录“用户已被撤销”并不自动意味着该用户无法继续获取链下密钥材料：如果密文对象与密钥材料在授权有效期间已经分发到用户侧，撤销只能阻止后续的新请求，而无法收回已经交付的密文；更重要的是，如果链下对象存储、密钥封装记录与链上授权状态之间缺乏版本关联，撤销、文件更新、密钥轮换等事件就无法传导到材料释放判定与密文对象版本上。因此，第三个问题是如何把授权状态、密文对象版本、密钥材料与数据库任务状态组织成可验证的闭合关系，实现前瞻性撤销与故障恢复。

综上，本研究面向非连续时间约束下的区块链数据共享场景，以“策略表示—可信授权执行—密文对象生命周期管理”为主线设置三项递进研究内容：其一解决非连续时间策略的确定性表示与编译，其二解决基于许可联盟链状态锚定的可信授权执行，其三解决授权状态变化后的版本化密文头部与前瞻性撤销闭环。三者共享半开区间时间语义与策略摘要，前一阶段输出的语义与状态作为后一阶段的输入，最终形成从策略编译、授权判定到材料释放与恢复的可验证闭合链路。

从现有技术路线看，访问控制研究已经发展出角色访问控制、属性访问控制与令牌授权等多种机制：角色与时间约束模型能够表达周期性角色启停等时间语义^{[8][9]}，属性基加密能够按属性实施细粒度访问控制^{[10][11][12]}，令牌授权框架能够把权限以可携带凭证的形式分发。然而，这些机制在面对“非连续时间约束+动态授权状态+链下密文生命周期”的组合场景时都存在结构性缺口：角色模型与属性策略通常把时间作为单一约束条件，难以表达由多个非对齐窗口与例外片段构成的规则；普通令牌的有效性取决于签发时点的快照^{[13][14]}，难以在验证时点约束动态状态与跨实例重放^{[13][14]}；属性基加密的长期密钥资格不能反映某次访问发生时的链上状态。区块链相关数据共享方案近年来得到广泛研究^{[15][16]}，但多数工作聚焦于链上存证、去中心化存储或属性加密的结合，较少把策略语义确定性、授权状态锚定与密文对象版本化作为一个连贯的问题链整体处理，这为本文方案提供了明确的研究空间。

密文对象治理方面，数据共享的最终载体是链下密文，其生命周期包括生成、封装、分发、更新、撤销与恢复多个阶段。现实业务中，授权状态变化与密文对象版本之间经常出现

失配：用户被撤销后，已分发的密钥材料仍可能继续使用；文件内容更新后，旧密文头与旧密钥封装记录可能仍指向已失效的版本；对象损坏或副本缺失时，恢复过程又可能因为缺少可信的完整性依据而引入新的风险。这些问题表明，数据共享方案不能只解决“谁可以访问”的问题，还必须解决“访问之后密文对象如何随状态演化”的问题，即把授权状态与密文对象的版本生命周期组织成统一的、可验证的闭合关系。

（2）研究目标与主要技术问题

本研究的总体目标是面向具有非连续时间约束和动态授权需求的区块链数据共享场景，建立一套安全边界清晰、协议接口闭合、能够通过原型与实验复核的可信共享方案。系统不追求把大文件或秘密计算迁移到链上，而是利用许可联盟链确定公开授权状态与请求顺序，利用能力结构与原子一次性机制实现跨实例一致的可信执行，并通过版本化密文头部把撤销与更新传导到链下材料释放。围绕这一目标，本研究需要回答三个相互关联的主要技术问题。

第一个问题是确定性表示问题：非连续时间约束如何获得确定、唯一、可复现的语义表示与策略摘要。该问题之所以是基础，是因为策略摘要构成后续授权执行与审计复核的公共输入。若不同组件对同一策略计算出不同摘要，则链上状态、能力绑定与材料释放判定都会失去可信基础；若摘要随输入序列的书写方式变化，则等义策略会对应多个状态，破坏审计一致性。因此，确定性表示必须建立在语义等价类之上，即所有表达同一允许时间集合的输入都映射到同一规范表示与同一摘要，同时表示规模与编译复杂度具有可解释的边界。

第二个问题是可信执行问题：确定的授权策略如何在许可联盟链环境中形成可信、可审计、抗重放的授权执行。该问题不能只依赖普通令牌或本地校验，原因在于授权状态是动态的：用户撤销、资源暂停、策略更新与密钥轮换都会改变当前请求是否仍然有效^[17]；令牌一旦签发，其有效性必须由验证时点的链上状态决定，而不能只由签发时点的快照决定。同时，系统存在多个验证实例，重复消费判定必须在实例之间形成统一语义，防止同一能力被并发地多次使用；能力还必须绑定到具体链、具体合约实例与具体资源状态，防止在不同部署环境之间迁移。

第三个问题是生命周期一致性问题：授权状态变化后，链下密文对象、密钥材料与链上状态如何维持版本一致性，并实现前瞻性撤销与故障恢复。该问题之所以最终必须传导到密文对象，是因为数据共享的最终载体是链下密文：授权状态变化只有反映到密钥封装记录、密文头部与材料释放判定上，才能真正改变用户的访问能力。撤销应当立即停止后续材料释放，并在新密文头部闭合后恢复合法用户的访问；故障场景下，对象损坏、摘要不一致与副本缺失都需要可验证的恢复路径，且恢复过程不得放宽完整性要求。

三个问题的关系是递进的：确定性表示提供语义与摘要基础，使链上链下各方对同一策略形成一致认知；可信执行把该语义与动态授权状态锚定到链上，使授权判定具备统一事实源与抗重放能力；生命周期一致性把链上状态传导到密文对象版本与材料释放，使授权结果真正落到数据访问本身。缺少任何一环，方案都会退化为“链上记录”与“链下执行”相互脱节的半闭合系统。因此，本研究以三个问题的完整闭环作为总体目标，并以“统一语义—状态

图1 论文总体技术路线与三项研究内容递进关系

(3) 研究思路、技术路线与主要创新点

本研究的技术路线可以概括为“一次数据共享的完整生命周期”：数据所有者首先以非连续时间窗口定义授权策略；策略编译组件对窗口进行离散化、规范化与确定性编码，生成唯一语义表示与策略摘要；资源与摘要信息注册到链上授权状态合约；数据用户获得由授权状态约束的能力；验证实例在验证时点重新读取链上状态、复核时间策略并原子消费一次性Nonce；材料释放组件依据链上复合状态决定是否释放密文材料；授权状态变化（暂停、撤销、更新、轮换）通过事件传导至密文头部更新与材料释放判定；对象损坏等故障通过恢复协调器从隔离副本验证并恢复。整个生命周期中，策略摘要、授权状态、任务状态与对象摘要相互绑定，构成可审计的闭合链路。

从系统组成看，方案包含数据所有者、数据用户、策略编译组件、许可联盟链（Besu QBFT）、链上授权状态合约、能力签发与验证组件、共享数据库、密文头部注册表、链下对象存储与隔离副本、材料释放组件与恢复协调器等部分。各部分并非简单的组件堆叠：策略编译组件输出的摘要进入链上状态与能力绑定；链上状态是签发与验证的唯一事实源；共享数据库提供跨实例的一次性消费；链下对象以内容摘要为完整性权威并受链上状态约束。系统不把大文件或秘密计算迁移到链上，链上只保存必要的公开状态、摘要与审计信息。

围绕上述技术路线，本阶段形成三项阶段性创新方向。

创新点一：面向非连续时间约束的确定性策略表示与编译。已有访问控制模型多面向连续时间或单一时间点，难以表达由多个非对齐窗口、例外日期与周期片段构成的授权规则；若将最小时间单元逐一写入策略，表示规模随时间域线性膨胀；而不同输入序列可能对应同一允许时间集合却产生不同摘要，破坏跨组件一致性。本文针对这些问题，首先把“策略语义”与“策略书写形式”解耦：以统一时区、最小粒度和半开区间定义时间域与策略语义，以规范区间序列作为唯一语义表示^[18]，使任何表达同一允许时间集合的输入都经规范化映射到同一表示；其次把“语义表示”与“派生执行表示”解耦：层次覆盖只作为可再生成的执行结构，不参与语义与摘要定义；最后以固定宽度规范编码与摘要计算把唯一语义固化^[19]为唯一标识，使策略编译、授权签发、验证复核与审计组件共享同一策略事实。与常规区间压缩方法相比，本文的关键区别在于摘要由语义表示决定而非由执行表示决定，从而保证了表示层变化不引起策略标识变化。阶段验证覆盖形式化模型、算法实现、性质测试与 15120 条正式记录^[20]，明确了规范区间在实验范围内的稳健性以及层次覆盖的适用边界。

创新点二：基于许可联盟链状态锚定的可信授权执行。已有方案或依赖中心化授权服务，存在单点失效与越权响应风险；或使用无状态令牌，无法在验证时点约束动态状态、无法统一跨实例重放语义、也无法防止能力在链与合约之间迁移。本文以许可联盟链为授权状态的事实源：资源状态与用户状态以版本化模型维护在链上，能力结构把链标识、合约地址、策略摘要、epoch、资源状态版本与用户密钥版本等维度完整绑定，每个绑定维度对应一类具

体风险；两个相互独立的验证实例共享数据库原子约束，以单条事务完成 **Nonce** 一次性消费，使“同一能力只能成功消费一次”在实例之间具有统一语义；验证流程在依赖故障时拒绝而非降级放行。与令牌方案相比，能力有效性由验证时点的链上状态决定而非签发时点；与中心化方案相比，状态变更具有可审计的公开记录。阶段实验在真实五节点链上完成，验证了重放控制、状态竞争控制与故障闭合，并明确了链读取主导时延这一性能边界。

创新点三：版本化密文头部与前瞻性撤销闭环。已有数据共享方案在授权撤销后通常只能阻止新请求，难以把撤销传导到已分发密文对象的生命周期；重新加密整个文件主体又带来与文件规模成正比的数据搬移开销。本文把密文对象组织为 **Header** 与 **Body** 两部分，以 **Header**、**Body** 与内容密钥的独立版本描述对象更新：授权语义变化通过仅更新 **Header** 完成，密钥与对象内容变化通过整体轮换完成，两类操作具有不同的成本结构与语义，不作等价比较；材料释放判定依据链上复合状态与对象完整性共同执行，撤销事件发生后立即停止后续材料释放，在新 **Header** 闭合后恢复合法用户访问；数据库任务状态机以操作标识保证链上写入幂等，链下对象以内容摘要为完整性权威，恢复过程从隔离副本验证并恢复而不放宽完整性要求。方案采用前瞻性撤销语义，明确不追回此前已合法获得的明文或旧密钥。阶段实验覆盖状态一致性、撤销窗口 **Fail-Closed**、规模影响与故障恢复，未观察到错误材料释放。

（4）研究内容一：非连续时间策略的确定性表示与编译

研究内容一要解决的是非连续时间约束的确定性表示问题。其来源在于，同一授权规则可以由多种输入形式表达：用户提交的原始策略可能包含乱序区间、重复区间、相交或嵌套区间，也可能把相邻区间拆分成多个片段。这些输入虽然表达相同的允许时间集合，但若直接参与摘要计算或授权判断，会产生不同的字节表示和不同的策略标识，破坏跨组件校验的一致性。因此，研究首先需要回答：如何从任意等义输入中确定性地恢复唯一语义，并使其可以被链上链下各方共同复核。

为此，本研究首先建立时间策略的形式化模型。设系统研究的真实时间域为 $[t_0, t_0 + U\Delta)$ ，其中 t_0 为可转换为 UTC 的时区感知起点， $\Delta > 0$ 为时间槽粒度， $U \in \mathbb{N}^+$ 为槽总数；离散时间域为 $T = \{0, 1, \dots, U-1\}$ ，任意真实时间点通过取整映射到槽坐标。策略以半开区间 $[l, r)$ 表达一段连续允许时间，原始策略是允许重复的区间序列 $P = \langle [l_1, r_1), \dots, [l_n, r_n) \rangle$ ，其语义定义为允许槽集合 $S(P) = \bigcup_{i=1}^n \{x \in T \mid l_i \leq x < r_i\}$ 。该模型以半开区间避免端点归属歧义，以统一粒度保证跨组件计算的确定性，为后续规范化与编码提供公共基础。

$$S(P) = \bigcup_{i=1}^n \{x \in T \mid l_i \leq x < r_i\}. \quad (1)$$

$$T = \{0, 1, \dots, U-1\}. \quad (2)$$

$$\phi(t) = \lfloor \frac{t-t_0}{\Delta} \rfloor. \quad (3)$$

$$\mathcal{D} = [t_0, t_0 + U\Delta), \quad (4)$$

在形式化模型之上，本研究定义唯一语义表示 I^* ：它是允许槽集合的极大连续分量分解，即把策略语义规范化为有序、互不重叠、互不相邻的规范区间序列。规范化过程 (**Normalize**) 首先按区间左端点排序，再进行一次线性扫描：若新区间与当前分量相交或相

邻，则合并为凸并；否则结束当前分量并开始新的分量。重复区间、嵌套区间与相邻区间由同一合并条件自然消除，输入对象不被修改，输出仅依赖允许槽集合而不依赖输入序列的书写方式。据此，任意等义输入都映射到同一 I^* ，从而获得语义上的唯一性。

算法 1 非连续时间策略规范化 (Normalize)

输入：已离散化区间序列 P ，时间域 $T=\{0,1,\dots,U-1\}$
 输出：规范区间序列 I^*

- 1: 将 P 中每个区间按左端点升序排序，同左端点按右端点升序
- 2: 初始化空分量序列 $cur \leftarrow []$
- 3: FOR EACH 区间 $[l,r) \in$ 排序后序列 DO
- 4: IF cur 非空 AND $l \leq cur.right$ 或 $l = cur.right$ THEN
- 5: $cur.right \leftarrow \max(cur.right, r)$ /* 合并相交或相邻区间 */
- 6: ELSE
- 7: 将 cur 加入 I ; $cur \leftarrow [l,r)$ / 开始新分量 */
- 8: END IF
- 9: END FOR
- 10: 将最后一个 cur 加入 I^*
- 11: 返回 I^* /* 有序、互斥、互不相邻 */

算法结束]

$$I^* = \text{Normalize}(P) = \langle [a_1, b_1), \dots, [a_k, b_k) \rangle. \quad (5)$$

除语义表示外，研究还构造了层次执行表示 $C(P)$ ：以二进制对齐节点（起点可被长度整除、长度为 2 的幂的半开区间）对规范区间进行最大分解。算法从左至右为每个未覆盖左端点选择当前位置可用的最大对齐块；若该块超过剩余长度，则持续右移缩小，直至完全位于源区间内。所得节点集合两两互斥、首尾相接且并集等于源区间，覆盖的节点总数记为 c 。 $C(P)$ 为后续可能的节点级授权、缓存键与聚合接口提供确定性的层次节点标识，但它不参与策略语义与摘要的定义。本文明确区分两种表示的职责： I^* 是主语义表示，负责表达语义、生成摘要与支持普通匹配； $C(P)$ 是派生执行表示，负责提供层次结构，且可以随时由 I^* 再生成。

算法 2 Dyadic 层次覆盖生成 (Cover)

输入：规范区间 $I=[l,r)$ ，槽总数 U ($L=2^{\lceil \log_2 U \rceil}$)
 输出：最大对齐覆盖节点集合 C

- 1: 初始化 $C \leftarrow \emptyset$, $pos \leftarrow l$
- 2: WHILE $pos < r$ DO
- 3: $size \leftarrow$ 当前位置可用的最大 2 的幂对齐块
- 4: WHILE $size > r-pos$ DO $size \leftarrow size \gg 1$ /* 不超过剩余长度 */
- 5: 将节点 $(pos, size)$ 加入 C

6: pc
7: END
8: 返回
算法结

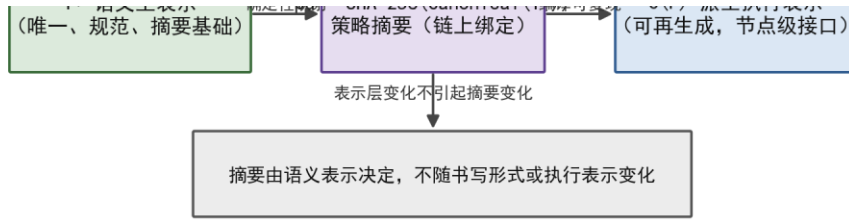


图2 语义主表示—摘要—派生执行 IR 关系

$$C(P) = \bigcup_{I \in I^*} C(I), c = |C(P)|. \quad (6)$$

$$D(j, s) = [j2^s, (j+1)2^s). \quad (7)$$

$$L = 2^{\lceil \log_2 U \rceil}. \quad (8)$$

为保证跨组件摘要一致，研究设计了 NTP1 固定宽度大端编码：规范字节串由 magic、schema、时区起点、粒度、时间域大小、区间数量与区间列表等字段按固定顺序和宽度编码，序列化器拒绝微秒未对齐起点、非整秒粒度、非法字段范围、越界或非规范区间；解码后重新调用编码器验证规范性，降低编码与校验规则分叉的风险。策略摘要定义为 $pd = \text{SHA256}(B(P))$ ，其中 $B(P)$ 为规范字节串。摘要绑定时间解释环境与 I^* ，而不绑定 $C(P)$ ，原因在于 I^* 直接表达策略语义，只要语义、起点、粒度和时间域不变，执行层更换索引或覆盖实现不应改变链上或协议中的策略标识。

$$pd = \text{SHA256}(B(P)). \quad (9)$$

上述流程的正确性建立在若干分析结论之上：规范化保持允许槽集合不变，且输出唯一；层次覆盖完整覆盖源区间且节点互斥；规范序列在固定时间解释环境下唯一；确定性哈希对相同字节输入输出相同摘要。研究同时给出输出敏感复杂度刻画：整体编译复杂度为 $O(n \log n + c)$ ，其中 n 为原始区间数， c 为覆盖节点数；该式表示复杂度由实际输出规模决定，不意味着任意非连续策略具有对数级表示。对于由 k 个彼此隔离的单槽构成的策略，任何保持独立分量的表示至少需要 $\Omega(k)$ 个信息单元，层次覆盖在该情形下同样达到 $c = k$ 。 $T(n, c) = O(n \log n + c)$ 。

算法层面，确定性编译流程由算法 1 给出整体组织：首先校验输入满足时区与粒度约束，随后依次执行时区归一化、区间规范化、层次覆盖构造、规范编码与摘要计算，最后返回语义表示、执行表示、规范字节串与摘要。规范化算法基于排序与线性扫描合并，输入顺序、重复区间与等价拆分不会改变输出；覆盖算法基于最大对齐块选择，通过位运算快速确定当前位置可用的最大二进制块，并保证节点互斥与完整覆盖。两段算法分别对应编译器与覆盖模块的实现，真实时间离散化在核心入口之前完成，使算法输入与实验使用的整数策略保持一致，也便于对核心逻辑进行独立测试与形式化核对。

算法3 确定性策略编译与摘要生成 (PolicyCompile

输入：时区感知起点 t_0 、时间粒度 Δ 、槽总数 U 、区间序列 P
 输出：唯一语义表示 I^* 、层次执行表示 C 、规范字节串 B 、策略摘要 pd
 1: 校验 t_0 可转换为 UTC、 $\Delta > 0$ 、 $U > 0$ 且 P 的端点均落在 $[0, U)$ 内
 2: $I^* \leftarrow \text{Normalize}(P)$ /* 唯一语义表示 */

3: C ←
4: B ←
5: pd ←
6: 返回
算法结

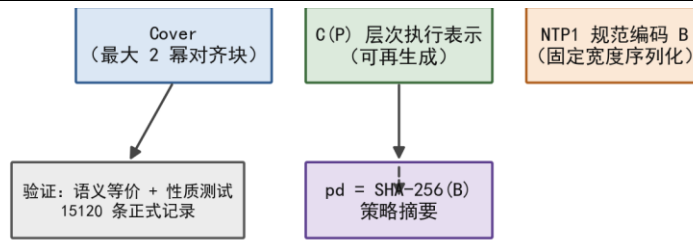


图 3 非连续时间策略确定性编译流程

NTP1 编码字段方面，规范字节串由固定头部与区间列表组成：头部包括标识、模式版本、UTC 起点、粒度、时间域大小与区间数量等字段，区间列表按 (left,right) 定长编码，整体为固定宽度大端字节流。编码长度等于固定头部加 $16k$ 字节，其中 k 为规范区间数。固定宽度的意义在于消除序列化层的不确定性：JSON 等文本格式会因键顺序、数字表示与空白差异产生不同字节，而固定宽度编码保证相同语义必定产生相同字节，从而保证摘要的确定性。解码器在解析后重新调用编码器进行规范性校验，进一步降低编码与校验规则分叉的风险。

$$B(P) = CanonicalSerialize(t_0, \Delta, U, I^*). \quad (11)$$

工程原型方面，研究完成了 Python 实现，包括时区感知离散化、规范化、最大层次覆盖、规范编码与摘要计算等模块，并提供与抽象定义一致的自动化测试。正确性验证完成 81 项测试（含性质测试与小域穷举），分支感知代码覆盖率为 98.61%，未发现偏离形式化定义的反例；测试覆盖语义一致性、规范化幂等性、输入置换不变性、等价拆分摘要一致性与覆盖结构约束等性质。数学分析证明编码前语义规范性与编译过程正确性，测试验证具体实现与抽象定义一致，两者相互补充但不相互替代。

实验设计方面，研究选择三种表示进行比较：时间槽枚举（以冻结集合存储全部允许槽，逻辑编码按每槽 8 字节计算，查询为期望常数时间）、普通规范区间列表（以 16 字节区间端点存储，二分查找查询）、二进制层次覆盖（以 (start,size) 节点存储，沿叶到根路径检查）。三种表示共享同一 I^* 、同一语义策略与同一查询集合，逻辑编码字节、对象深层驻留内存与峰值分配分别统计，不混为同一指标。正式实验 E1 分为三部分：E1-A 覆盖时间域长度 $U \in \{10^3, 10^4, 10^5, 10^6\}$ 、目标碎片率 $F \in \{0, 0.5, 1\}$ 、目标覆盖率 $\rho \in \{0.01, 0.10, 0.50\}$ 、冗余度 $R = 2$ 与 3 个随机种子，共 108 个样本；E1-B 固定时间域与覆盖率，考察冗余度 $R \in \{1, 2, 4, 8\}$ ，共 36 个样本；E1-C 覆盖 8 类边界策略与 3 个种子，共 24 个样本。每个样本预热 5 次、正式重复 30 次，正式实验共 15120 条记录且全部有效。

表 1 三种表示的理论与实现特征

表示	构造复杂度	查询复杂度	E1-A 样本逻辑字节中位数	E1-A 查询中位数 /ns	主要用途
时间槽枚举	$O(A)$	期望 $O(1)$	24000	350.4	小域、高频查询
规范区间列表	$O(n \log n)$	$O(\log k)$	2808	561.0	一维存储与匹配
层次覆盖	$O(n \log n + c)$	$O(\log U)$	3664	1984.7	层次授权接口

阶段性实验结果表明：在统一定长编码下，层次覆盖相对普通规范区间列表更紧凑 0

次、相同 36 次、更大 72 次，未观察到层次覆盖比规范区间列表更紧凑的样本；原因是每个非空规范

查询性能方
350.4 ns、56
进一步表明
次执行结构

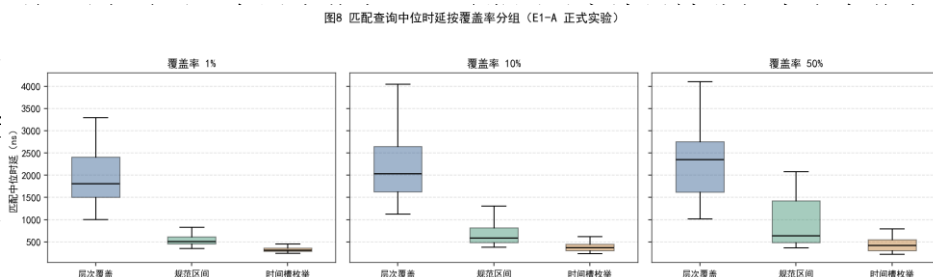


图 4 匹配查询中位时延 (E1-A 正式实验结果)

从规模数据看，E1-A 样本逻辑字节中位数分别为：槽枚举 24000 字节、规范区间列表 2808 字节、

区间与覆盖
片时，规范
示的规模关
范化处理量
证直接检查
数槽、奇数
策略下层次

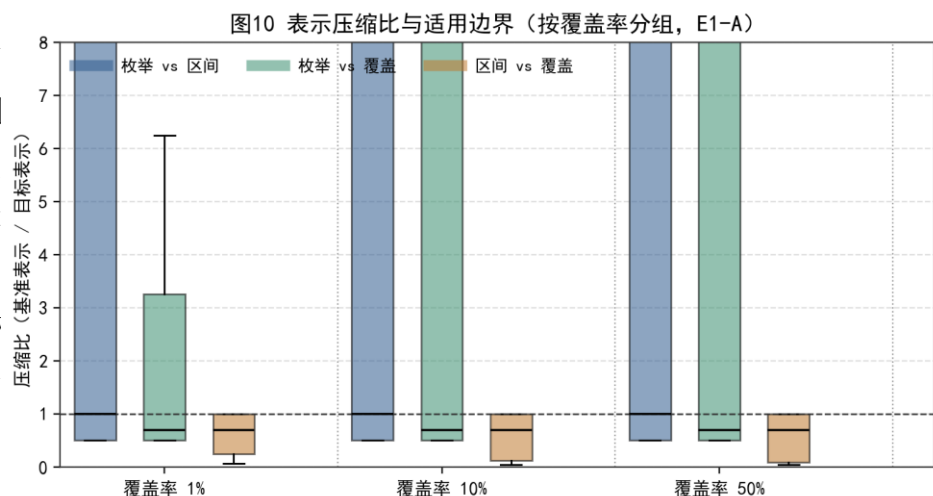


图 6 表示的压缩比与适用边界 (E1-A 正式实验结果)

负结果的解读是研究内容一的重要收获：层次覆盖没有表现出相对规范区间列表的普遍存储优势，这并不意味着设计失败，而是把“覆盖结构是否有价值”这一假设从“存储压缩”转向“协议接口”：在保留 I^* 主表示的同时， $C(P)$ 可以作为节点级授权、缓存键与聚合接口的输入，其价值需要由后续授权协议来检验。这一认识直接约束了研究内容二的设计空间，也防止把未经验证的性能假设写入论文结论。

综上，研究内容一目前已完成时间策略形式化、唯一语义表示、确定性编译流程、正确性分析与输出敏感复杂度刻画，并完成 168 个样本、15120 条记录与 81 项测试的阶段验证。现有证据支持 I^* 作为主语义表示与摘要基础，同时明确表明层次覆盖不具普遍存储或一维查询优势。因此，后续研究不再将层次覆盖作为压缩优势的来源，而将其定位为派生执行结构与节点级授权接口的可选输入；这一负结果构成方法适用边界的一部分，也为研究内容二的授权执行设计提供了明确的输入约束。

(5) 研究内容二：基于许可联盟链的可信授权执行

研究内容二要解决的是授权执行的可信性问题。研究内容一输出的唯一语义表示与策略摘要构成研究内容二的输入：资源在链上注册时记录策略摘要，能力签发与验证都绑定该摘要，验证流程重新执行时间策略检查，从而把策略语义、策略摘要与链上状态绑定在一起。但仅有策略表示仍然不够——授权执行面对的是动态状态：用户可能被暂停或撤销，资源策

即 $c \geq k$ 。

数分别约为
幕全域实验
收益属于层

较大而规范
趋于单槽碎
小，三种表
区间数与规
由正确性验
实验覆盖偶
确认碎片化

略可能更新
单一服务端
被重复消费
基于上
链相比，许
景；与中心
适合作为多
Validator 节
标识固定，
提供确定性

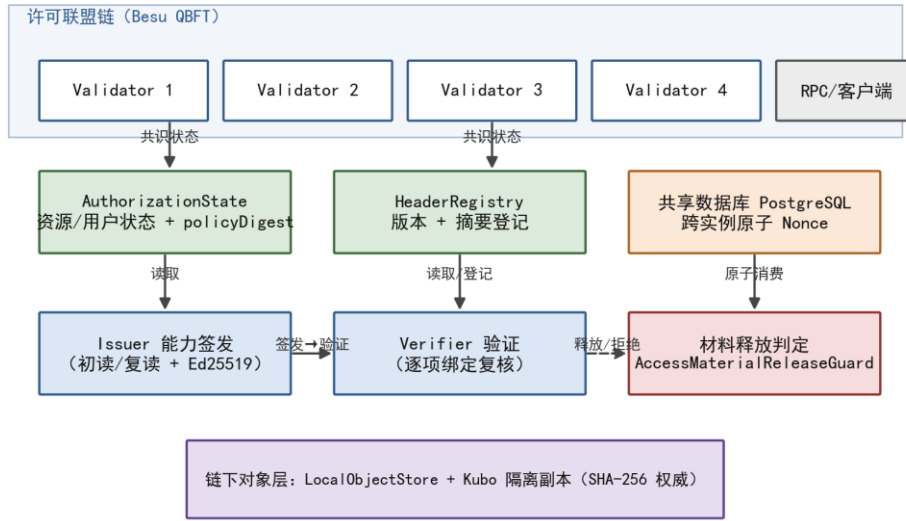


图7 许可联盟链可信授权系统总体架构

链上授权状态由 **AuthorizationState** 合约分别维护资源状态与用户状态。资源状态记录所有者、策略摘要、epoch、状态、策略版本、状态版本与更新时间；用户状态记录账户、用户密钥标识、状态、用户版本与更新时间。状态枚举为 **NONE**、**ACTIVE**、**SUSPENDED** 与 **REVOKED**：资源注册后进入 **ACTIVE**，策略更新递增策略版本与状态版本，epoch 推进递增 epoch 与状态版本，暂停、恢复与撤销推动资源状态及状态版本；用户注册后进入 **ACTIVE**，密钥轮换递增用户版本，用户状态变化同样递增用户版本；资源或用户一旦进入 **REVOKED** 不得恢复为 **ACTIVE**。版本字段构成状态快照的基础：任何验证与材料释放判定都可以基于某一时刻的快照进行，并通过版本比较检测状态变化。

$$stateVersion' = stateVersion + 1 \wedge REVOKED \text{ 为终态.} \quad (12)$$

$$U = (account, userKeyId, status, userVersion, updatedAtBlock). \quad (13)$$

$$R = (owner, policyDigest, epoch, status, policyVersion, stateVersion, updatedAtBlock). \quad (14)$$

在授权状态模型之上，研究设计了 **CAP2** 能力结构。设计的动因是：普通令牌通常只包含签发者、主体与有效期，无法约束令牌在哪个链、哪个合约实例、哪个资源状态版本下有效；持有令牌的用户可以把令牌用于其他环境或其他资源，也可以在状态变化后继续使用旧令牌。**CAP2** 以规范化字节序列为输入，使用 **Ed25519** 签名，将链标识、合约地址、策略摘要、epoch^[22]、资源状态版本、用户密钥标识与用户版本、操作类型、生效与失效时间以及一次性 **Nonce** 等字段完整绑定。其中，链标识与合约地址阻止能力迁移到其他链或合约实例；策略摘要、epoch 与资源状态版本绑定资源当前状态；用户密钥标识与用户版本绑定当前用户密钥；操作类型、生效与失效时间限定用途与时间窗口；**Nonce** 支撑一次性消费。每个绑定维度对应一类具体的失效风险，验证时对全部维度逐一复核。

$$\sigma = Ed25519.Sign(sk_I, B). \quad (15)$$

$$B = Encode_{CAP2}(F_1 || F_2 || \dots || F_n). \quad (16)$$

编码层面，**CAP2** 采用规范化字节序列作为签名输入：整数采用无符号大端编码，变长字符串带 16 位长度前缀，字段顺序固定，从而保证同一能力内容对应唯一字节串，签名结

果具有确定性。策略摘要始终由 I^* 计算, $C(P)$ 不参与语义身份定义; 在启用层次模式的配置下, 能力可附加匹配节点与覆盖版本字段, 供后续节点级授权实验使用, 但这些字段不改变语义身份。签名密钥由 Issuer 持有并保护, 验证方通过登记的公钥验签, 把能力内容的完整性与签发者的可信性绑定。

能力签发流程由 Issuer 执行: 首先在同一确认区块读取资源状态与用户状态, 检查资源与用户均为 ACTIVE; 随后校验用户公钥哈希与链上密钥标识一致, 检查策略摘要一致且当前时间落入策略允许窗口; 生成 Nonce、有效期与待签字段后, 在签名前再次读取同一资源与用户状态, 若两次快照不一致则拒绝签发。签名前复读的目的在于防止“签发时点与读取时点之间状态已经变化”的竞态: 签发操作依据的是签名前的最终确认快照, 避免把基于过期状态的判断固化到能力中。

算法 4 CAP2 能力签发 (Issue)

输入: 授权请求 (资源、用户、操作)、链上确认状态

输出: 签名能力 CAP2 或拒绝码

- 1: 在确认区块读取资源状态与用户状态
- 2: 校验资源与用户均为 ACTIVE, 且 policyDigest 与注册一致
- 3: 校验 $\text{SHA-256}(\text{用户公钥}) = \text{userKeyId}$, 且当前时间落入策略允许窗口
- 4: 生成一次性 Nonce、生效与失效时间, 组装待签字段
- 5: 签名前在同一确认区块复读资源与用户状态
- 6: IF 两次快照不一致 THEN 返回 REJECT /* 防止签发时点竞态 */
- 7: 规范化编码并以 Ed25519 私钥签名, 返回 CAP2

算法结束]

验证流程由 Verifier 执行: 解析并验签后, 读取确认链状态, 依次检查资源状态、用户状态、策略摘要、epoch、链与合约绑定、状态版本、用户版本、操作类型与时间窗口, 并重新执行时间策略检查; 全部通过后, 原子消费共享 Nonce, 冲突则返回重放拒绝, 仅当全部检查通过且消费成功时返回接受。验证阶段重新读取链上状态而非信任签发时的快照, 原因在于授权状态可能在签发与使用之间发生变化: 用户可能已被撤销、资源可能已被暂停、密钥可能已经轮换, 只有验证时点的链上状态才能反映当前是否仍然有效^[23]。

算法 5 CAP2 验证与共享 Nonce 消费 (Verify)

输入: CAP2 能力、请求上下文

输出: ACCEPT 或对应拒绝码

- 1: 解析规范编码; 失败返回 MALFORMED_TOKEN
- 2: 验证 Ed25519 签名; 失败返回 INVALID_SIGNATURE
- 3: 读取确认链上状态; 失败返回 SYSTEM_STATE_UNAVAILABLE
- 4: 逐项复核: 资源/用户状态、policyDigest、epoch、链与合约绑定、版本、操作与时间窗口

窗口

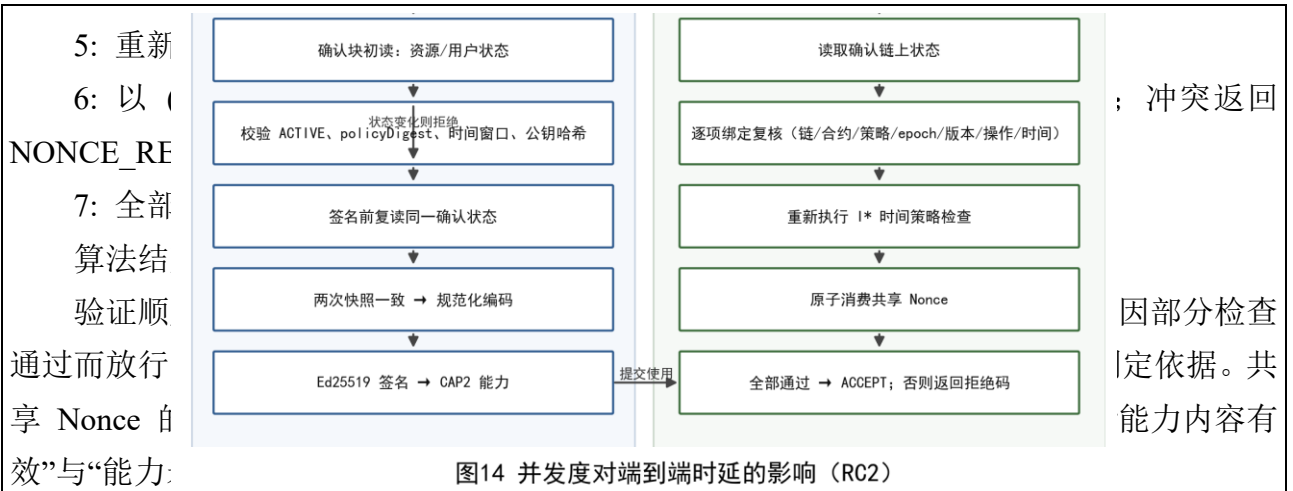
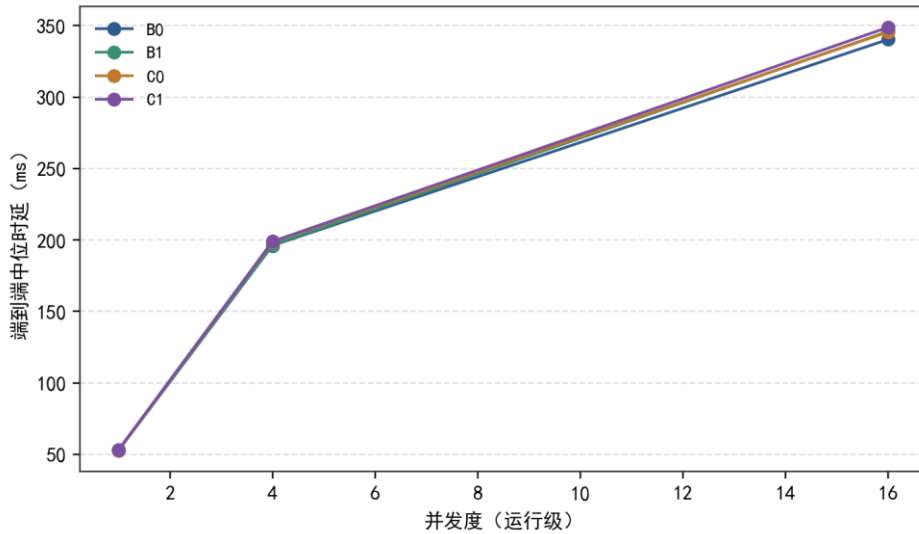


图14 并发度对端到端时延的影响 (RC2)

跨实例
者不共享进
“一次性”语
Nonce: 以
INSERT ...
示本次请求
内状态。并
次重放，验



Verifier, 二
用; 必须把
现共享原子
, 通过单条
返回一行表
回退到进程
、99 与 499

图 9 并发度对端到端时延的影响 (RC2)

$\text{INSERT}(k)$ returns $1 \Leftrightarrow k \notin \text{consumed}, k = (\text{chain}, \text{contract}, \text{resource}, \text{epoch}, \text{nonce})$. (17)

故障闭合方面，系统明确依赖故障时的行为：当 RPC 或数据库不可用时，系统拒绝授权而不是降级放行。数据库中断期间 Verifier 保持 Fail-Closed，恢复后既有消费记录仍可阻止重放；验证流程中任何一次状态读取失败都返回系统状态不可用，不把不完整信息作为判定依据。信任边界方面，系统采用角色分离：ADMIN 管理初始角色，OWNER 登记资源并更新策略，AUTHORIZER 推进 epoch，REVOCATION 管理暂停与撤销，AUDITOR 只读审计；角色分离降低单一业务账户承担全部权限的风险，但不阻止已获得合法管理角色的主体恶意操作，此类主体仍属于治理信任边界。安全验证方面，攻击回归、语义一致性、并发重放与状态竞争测试均未出现错误接受；这些证据支持冻结实现与故障模型下的系统主张，但不构成形式化安全证明，也不支持抵抗任意 Validator 合谋或追回已被合法用户取得的明文。

$\text{release} \Rightarrow \text{status} = \text{ACTIVE} \wedge \text{dbAvailable}$. (18)

实验环境方面，正式实验使用独立的正式链而非冷保留的基础设施验证链：正式链采用 Besu 26.5.0、Java 21、QBFT 共识、四个 Validator 与一个非验证 RPC 节点，链标识与网络标识固定，Genesis 摘要与合约工件摘要均在实验前冻结；五台 Ubuntu 24.04 虚拟机运行于 Windows 主机的 VMware 环境，客户端虚拟机承载非验证 RPC、Issuer、两个 Verifier 与

PostgreSQL。正式证据冻结了节点角色、软件版本、链参数与服务健康状态；由于单台虚拟机的精确配额未形成独立清单，论文不据此作硬件效率外推，并将共享物理主机列为有效性限制。第一次正式运行曾因链读取边界、局部性生成、缓存命中记录、吞吐量与统计单位等协议偏差被标记无效并重新预注册重跑，最终结论仅基于完整有效的 V13 重跑结果。

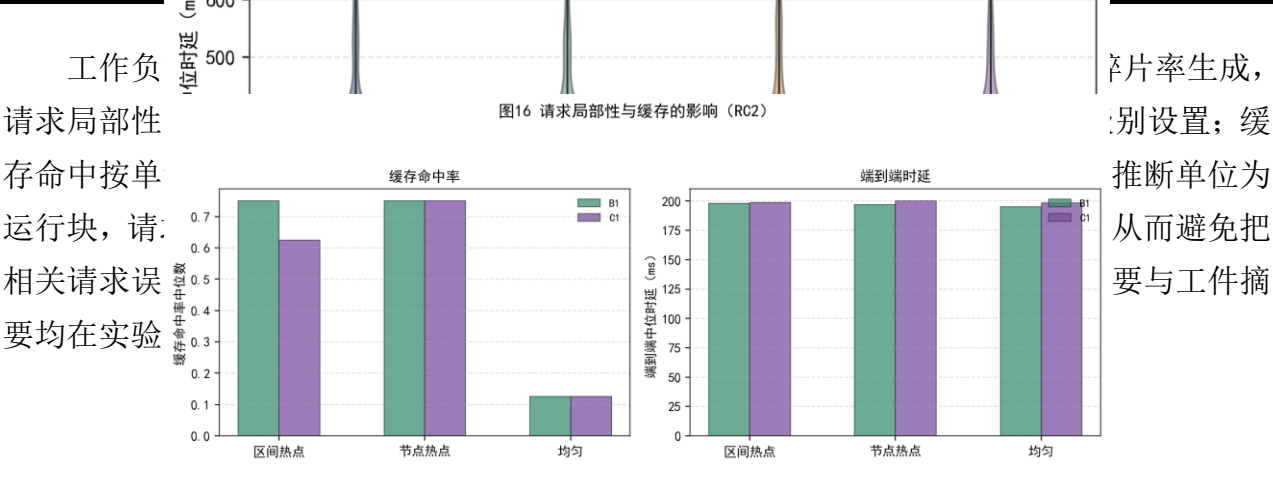
表 2 系统安全目标、机制与证据及结论边界

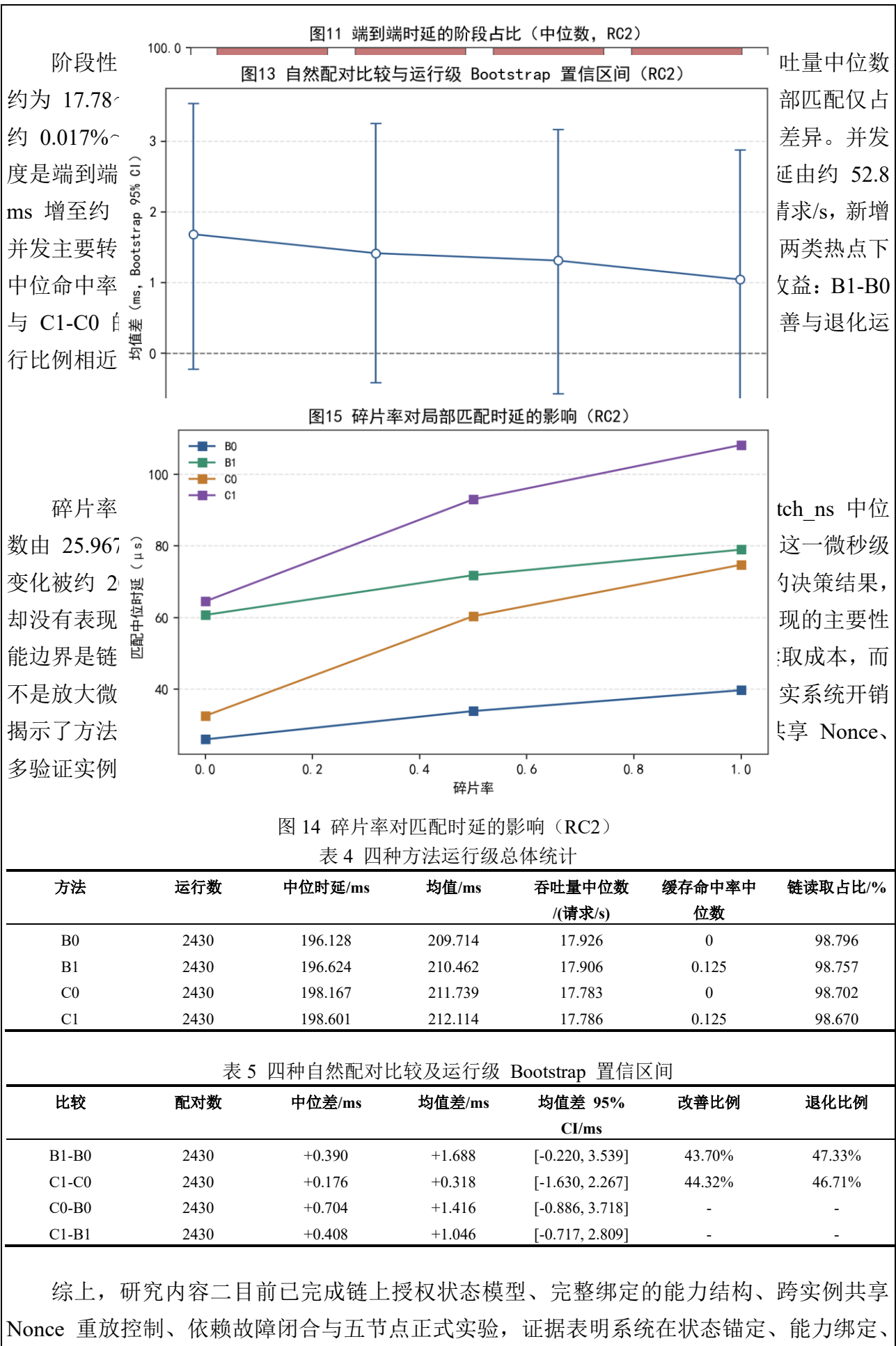
目标	机制与证据	结论边界
S1 状态锚定与策略绑定	资源/用户状态上链, policyDigest 绑定	不证明链本身绝对可信
S2 能力完整绑定	CAP2 绑定链/合约/策略/版本, 篡改测试拒绝	不抵抗合法角色恶意使用
S3 跨实例重放控制	共享原子 Nonce, 并发 50/100/500 均仅一次成功	限于冻结命名空间与事务语义
S4 依赖故障闭合	RPC/数据库故障时拒绝授权	不构成形式化安全证明

正式实验在五节点真实链环境上完成。环境包括四台 Validator 虚拟机与一台承载非验证 RPC、Issuer、两个 Verifier 与 PostgreSQL 的客户端虚拟机，链参数与软件版本在实验前冻结。实验比较四种方法：B0 为无缓存的基线，B1 为区间缓存，C0 为无缓存的层次覆盖执行，C1 为层次节点缓存；自变量包括三种碎片率（0、0.5、1）、三种请求局部性（均匀、区间热点、节点热点）与并发度（1、4、16），采用三个固定随机种子，每个含 seed 配置进行 30 次正式重复。实验共覆盖 108 个因素配置、324 个含 seed 配置、9720 个运行块、77760 条请求记录与 233280 条链读取记录，每次请求执行三次真实链读取；方法比较在相同工作负载、seed 与重复下自然配对，每组比较含 2430 对运行块，并执行 10000 次运行级 Bootstrap^[26]。

表 3 正式实验因素设计汇总

因素/规模	水平或取值	配置数	说明
碎片率	0 / 0.5 / 1	3	策略碎片程度
请求局部性	均匀 / 区间热点 / 节点热点	3	工作负载生成器
并发度	1 / 4 / 16	3	运行级并发
随机重	图12 四种方法的运行级端到端时延分布（RC2 正式实验）		
请求/配对			
工作负载			





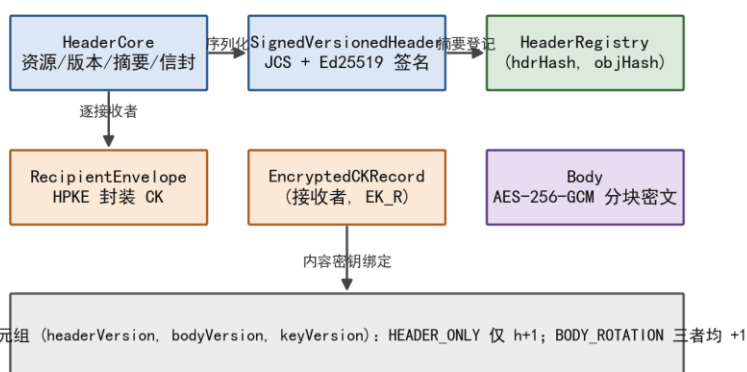
重放控制与故障闭合方面形成了可审计的授权执行机制。同时，阶段性实验也明确表明：缓存与层次覆盖在当前冻结工作负载下未产生稳定的端到端工程收益，C(P) 未表现出基线难以复制的协议能力或性能价值。因此，后续研究不再把缓存优化或 C(P) 性能作为核心贡献，而将研究重点收敛至链上状态锚定、完整绑定、共享 Nonce、多验证实例一致性与 Fail-Closed，并据此完成技术路线的收窄与确认。

（6）研究内容三：版本化密文头部与前瞻性撤销闭环

研究内容三要解决的是授权状态与链下密文对象生命周期的一致性问题。研究内容二完成后，链上已经能够维护可信的授权状态并控制授权判定，但数据共享的最终载体是链下密文对象与密钥材料：授权通过后，密文材料是否释放、释放后对象是否随状态变化更新、撤销后旧材料是否仍可被继续使用，这些环节仍未闭合。链上记录“用户已被撤销”并不自动意味着用户无法继续获取链下密钥材料；如果密文对象与密钥封装记录与链上状态缺乏版本关联，撤销、文件更新、密钥轮换等事件就无法传导到材料释放判定与密文对象版本上。因此，研究内容三把“授权状态—密文对象版本—密钥材料—数据库任务”组织为可验证的闭合关系。

图6 版本化密文对象结构与生命周期关系

密文对象主体^[27]；Body 摘要使用 HPKE (Body 版本规范序列化记录，从而把致上，任何



密文，承载文/Key 版本、按接收者使用 record，每个 r 使用 JCS y 的提交记到摘要不一

图 15 版本化密文对象结构 (Header/Body/CK)

$$hdrHash = SHA256(Canonical(Header)), HeaderRegistry \leftarrow (hdrHash, objHash). \quad (19)$$

$$EK_R = HPKE.Seal(pk_R, CK). \quad (20)$$

$$C_{body} = AES-256-GCM(K, N, M). \quad (21)$$

$$keyVersion = bodyVersion. \quad (22)$$

HeaderCore 的字段设计遵循“对象自描述、摘要可核验”的原则：资源标识指向链上资源，版本三元组描述 Header/Body/CK 的当前版本，Body 摘要与接收者信封分别保证对象完整性与按接收者封装；签名覆盖规范化序列化的全部字段，任何字段被修改都会导致验签失败。接收者信封支持多接收者场景：每个接收者对应一个 HPKE 封装记录，接收者通过自己的密钥解封装获得内容密钥，未列入信封的实体无法获得 CK，从而实现“授权列表之外的接收者不可解密”。Body 采用分块加密便于流式处理与部分校验，分块数与摘要计算在发布时确定，后续轮换生成新 Body 与新 CK，旧版本仍可按摘要追溯。

需要说明的是，密文头部与密文主体的分离是降低动态更新成本的关键：文件主体通常

体积较大，重新加密会产生与文件规模成正比的数据搬移开销；而密文头部体积小、只承载版本与摘要信息，更新头部即可表达授权语义变化。该设计使 HEADER_ONLY 操作能够以接近固定成本完成，BODY_ROTATION 操作则只在密钥或内容确实需要更换时才执行，两类操作的边界与成本结构在实验中分别刻画。

版本语义被冻结为三类操作：INITIAL 表示初始状态 (headerVersion=1, bodyVersion=1, keyVersion=1)；HEADER_ONLY 表示仅更新 Header (headerVersion 加 1, bodyVersion 与 keyVersion 不变, Body 与 CK 不变)；BODY_ROTATION 表示整体轮换 (headerVersion、bodyVersion、keyVersion 均加 1, 生成新 CK 与新 Body)。HEADER_ONLY 用于授权语义变化（如撤销后的 Header 闭合）而不更换数据密钥，BODY_ROTATION 用于更换密文对象与密钥。二者是不同语义的操作：HEADER_ONLY 面向授权状态变化，BODY_ROTATION 面向密钥与对象更新，实验中分别分析，不作等价比较。

算法 7 BODY_ROTATION 流

输入：需要轮换的密文对象（密钥或内容变化）
 输出：新 Body、新内容密钥 CK'、新 Header
 1: 生成新内容密钥 CK'
 2: 使用 CK' 对 Body 进行 AES-256-GCM 分块加密
 3: 为每个接收者以 HPKE 生成新 EncryptedCKRecord
 4: 构造新 Header: (headerVersion, bodyVersion, keyVersion) $\leftarrow$ (h+1, b+1, k+1)
 5: JCS 序列化、Ed25519 签名并登记 HeaderRegistry
 算法结束]

算法 6 HEADER_ONLY 更新流

输入：受影响资源、授权语义变化（撤销/暂停/策略更新）
 输出：新 Header 与链上登记记录
 1: 解析受影响资源，生成 Header 更新意图
 2: 构造新 Header: headerVersion $\leftarrow$ h+1, bodyVersion 与 keyVersion 保持不变
 3: JCS 规范序列化并以 Ed25519 签名
 4: 将 (hdrHash, objHash) 登记至 HeaderRegistry
 5: Header 进入 current 后恢复合法材料释放；不更换数据密钥
 算法结束]

$$(h, b, k) \mapsto (h + 1, b + 1, k + 1). \quad (23)$$

$$(h, b, k) \mapsto (h + 1, b, k). \quad (24)$$

版本状态机由 AuthorizationState 与 HeaderRegistry 共同维护：AuthorizationState 保存 policyDigest、epoch、stateVersion 与资源状态，HeaderRegistry 保存 headerVersion、bodyVersion、keyVersion、摘要与操作身份；二者通过资源标识关联，形成“资源状态—对象版本”的双通道记录。任何状态更新必须满足版本单调性与摘要一致性：headerVersion、

bodyVersion、keyVersion 只增不减，Header 摘要与对象摘要与链上登记一致，否则更新被拒绝。这种双通道设计使撤销、文件更新、密钥轮换与时间策略变化都可以表达为版本递增与状态迁移，并为材料释放判定提供可复核的输入。

数据库控制面是链上写入与链下任务之间的同步层：任务在显式提交后即可被独立连接读取，经写入准入后广播交易，以回执与固定区块状态验证后固化为已提交；数据库事务不跨链回执等待，避免把链上确认时延引入数据库事务边界。operationId 保证重复执行幂等：同一任务即使因网络重试被执行多次，也只产生一次链上效果；提交结果不确定等异常按预注册规则处理，不擅自回滚链上已确认的状态。任务状态机的状态迁移覆盖创建、准入、广播、已提交与失败等路径，为审计与恢复提供完整的操作记录。

数据库控制面解决的是“链上写入与链下应用之间的一致性”问题：应用先写入任务记录再提交链上交易，链上回执与固定区块状态验证通过后才把任务固化为已提交，任何中间失败都保留可审计的记录；操作标识保证幂等，使重试不会产生重复效果。这一设计使链下任务状态与链上交易结果在最终一致的前提下可追踪，为撤销流程、Header 更新流程与恢复流程提供统一的状态基础。

材料释放判定由 AccessMaterialReleaseGuard 依据链上复合状态与 Header 对象完整性执行：只有资源与用户均为有效状态、Header 存在且摘要一致、当前时间落在策略允许窗口内时才允许释放。链上 HeaderRegistry 保存 headerVersion、bodyVersion、keyVersion、摘要与操作身份，与 AuthorizationState 共同构成状态事实源；数据库控制面以任务状态机管理链上写入：任务显式提交后可被独立连接读取，经准入后广播交易，以回执与固定区块状态验证后固化为已提交；数据库事务不跨链回执等待，operationId 保证重复执行幂等，提交结果不确定等异常按预注册规则处理。

release iff $status = ACTIVE \wedge t \in S(I^*) \wedge hdrValid$. (25)

撤销采用前瞻性语义：撤销事件发生后，系统立即停止后续材料释放；在新 Header 闭合前，合法用户可能暂时不可访问，这是 Fail-Closed 的设计代价。撤销流程为：链上 epoch 推进触发事件，事件扫描与受影响资源解析生成 Header 更新意图；在 Header 闭合前，材料释放判定保持拒绝；Header 进入当前状态后，合法材料恢复释放。系统不主张追溯撤销：不能收回此前已合法获得的明文、旧 CK 或旧密文，这一边界在安全分析中明确陈述，避免把撤销机制表述为能够追回已泄露数据。

链下对象层由 LocalObjectStore 与隔离的 Kubo 副本组成：LocalObjectStore 以不可变方式存储对象，写入原子，SHA-256 内容寻址为完整性权威^[29]；Kubo 仅作为隔离副本定位，CID 不替代 SHA-256 的完整性权威。恢复由 RecoveryCoordinator 协调：读取候选对象、SHA 验证、结构验证、原子恢复，最终形成一致状态或 Fail-Closed 结果。受控故障包括对象损坏、CID 不一致与本地/副本同时缺失等类型；恢复路径不因来源不同而放宽完整性要求。

算法 8 RecoveryCoordinator 故障恢

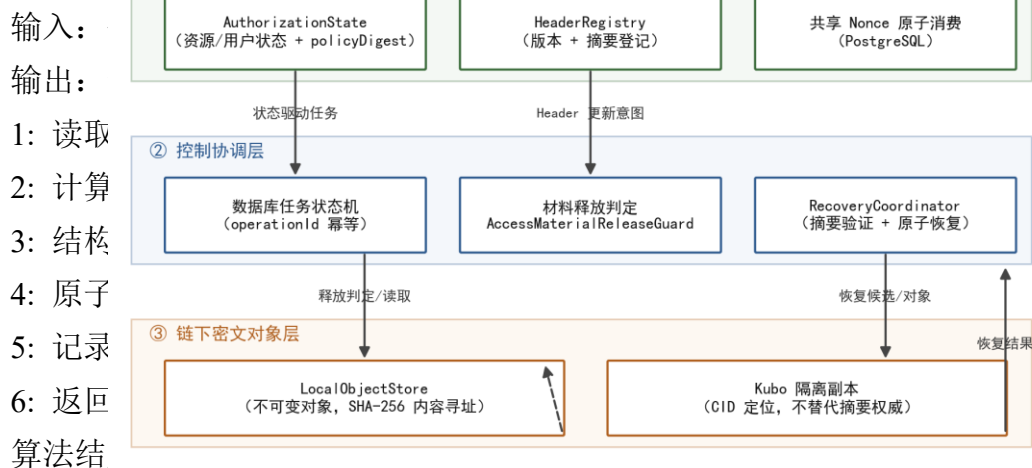


图 16 链上可信状态—控制协调—链下密文对象三层闭环架构

$\text{restore iff } \text{SHA256}(\text{candidate}) = \text{objHash} \wedge \text{structuralValid}.$ (26)

恢复协调的核心原则是“完整性权威唯一、恢复路径可验证”：无论候选对象来自本地存储还是隔离副本，都必须通过相同的 SHA-256 摘要验证与结构验证，验证通过后才能执行原子恢复；验证失败的对象不进入可用状态，系统保持 Fail-Closed 而不是降级使用不可信对象。恢复动作记录修复来源与修复数量，为审计提供依据。隔离副本采用独立 IPFS_PATH 且不连接公网 peer，避免副本被外部内容污染；副本与本地对象通过同一内容标识关联，但内容标识仅用于定位，完整性始终以 SHA-256 摘要为准。

正式实验在独立于 Pilot 的受控单节点环境中完成，环境包括独立 PostgreSQL 集群、隔离的 Kubo 仓库（零公网 peer）与独立单节点 QBFT 链，软件版本与链参数在实验前冻结；该环境仅用于应用层功能与受限工程测量，不用于评估多 Validator 共识性能。实验按冻结预注册执行：29 个配置、每配置 5 次重复、145 个 measured RUN（另有 35 个 warm-up 不计入统计），执行顺序由固定随机种子分块确定性随机化并在采集前冻结；统计以 RUN 为单位报告中位数与 IQR，使用 10000 次 Bootstrap 的 95% percentile 置信区间，比较采用中位差、比值与 Cliff's delta。

表 6 正式实验配置与运行汇总

实验	路径/变量	有效运行数	核心检查	结果
E1	INITIAL/BODY_ROTATION/REVOCATION/RESTORE	20	状态一致与幂等	全部通过，错误材料释放 0
E2	HEADER_ONLY/BODY_ROTATION 成本	30	版本关系	HEADER_ONLY 接近固定成本
E3	撤销窗口判定	15	Fail-Closed	pending 拒绝、闭合后允许
E4	pending/闭合路径	10	材料释放判定	拒绝/允许边界可追踪
E5	来源×故障类别	40	恢复判定	损坏场景副本可恢复，其余 Fail-Closed

五个实验分别验证不同侧面。E1 覆盖 INITIAL、BODY_ROTATION、REVOCATION

与 RESTORE 四种路径，共 20 个 RUN，状态一致性与幂等性检查全部通过，链/数据库/对象最终状态与 3147 ms 共 30 个 RUN 数差约 27.1 (1.002)，说明 E3 覆盖 Body 由 64 KiB 增至 8 KiB，Cliff's delta 全部正确。

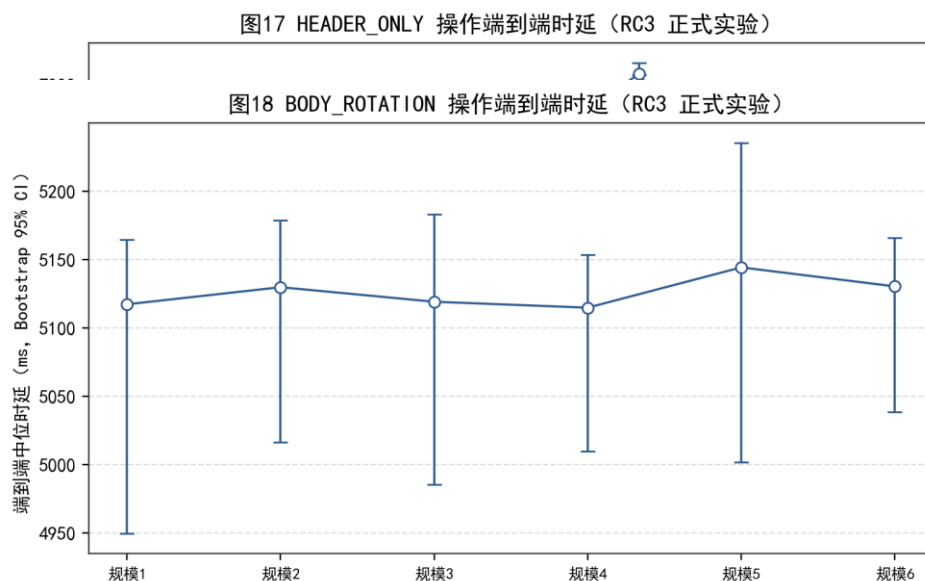


图 18 BODY_ROTATION 操作端到端时延 (RC3 正式实验结果)

从结果解释看，E1 的四种路径覆盖了对象从发布到恢复的主要阶段：INITIAL 建立对象与初始版本，BODY_ROTATION 执行密钥与对象轮换，REVOCATION 验证撤销后材料释放被拒绝，RESTORE 验证从副本恢复的一致性；四种路径的时延差异主要来自链上交易等待与固定流程，而非对象内容的差异。E2 与 E3 分别测量“仅更新 Header”与“整体轮换”两类操作的规模敏感性：E2 中接收者与受影响资源规模对时延影响较小，说明 HEADER_ONLY 的成本以固定链上流程为主；E3 中 Body 规模增大带来可观察的时延上升，说明 BODY_ROTATION 的成本与对象内容规模密切相关。

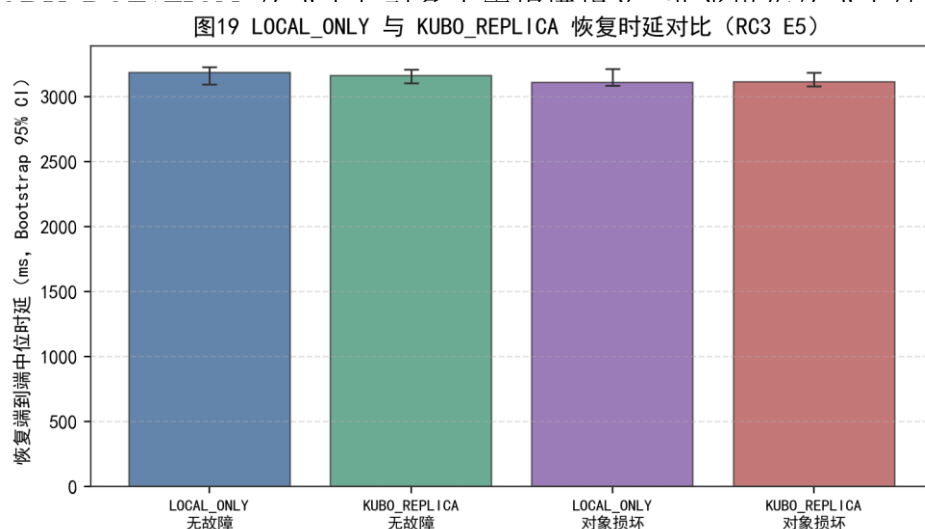


图 19 LOCAL_ONLY 与 KUBO_REPLICA 恢复时延对比 (RC3 E5)

从结果解释看，E5 的结果同时体现了恢复机制的收益与边界：副本机制的价值集中在“本地对象损坏但隔离副本完好”这一特定场景，它提供了可验证的恢复来源；而在无故障、CID 不一致与副本同时缺失场景下，副本不改变判定结果，系统按 Fail-Closed 处理。多数匹配块内 LOCAL_ONLY 与 KUBO_REPLICA 的 Cliff's delta 约 0.04，未观察到清晰的正常路径性能效应，因此副本被定位为可用性收益与恢复成本之间的 trade-off，而非性能增强。

手段。这些解释与“只描述、不夸大”的原则一致：恢复机制证明的是可验证性与边界，而不是普遍的性能优势。

研究内容三的适用范围被明确限定：单节点 QBFT 环境，不评估多 Validator 共识性能；29 个冻结配置与 5 次重复构成有界工程精度样本，不作总体推断；受控隔离环境，故障类别覆盖冻结的四类对象故障；仅前瞻性撤销，不主张追溯撤销；Body 规模与接收者规模范围有限；实验验证不构成形式化证明。这些限定与论文正文保持一致，任何超出范围的表述都不属于本阶段结论。

E4 覆盖撤销后的 pending 窗口与 Header 闭合两条路径，共 10 个 RUN：pending 窗口 5 次释放判定均为拒绝，Header 闭合后 5 次均为允许，错误材料释放为 0，撤销事件触发后状态转换可追踪。E5 覆盖两种对象来源（LOCAL_ONLY 与 KUBO_REPLICA）与四类故障（无故障、对象损坏、CID 不一致、本地与副本同时缺失），共 40 个 RUN：对象损坏场景下 LOCAL_ONLY 无法从其他来源恢复，结果为不可恢复（Fail-Closed，5/5）；KUBO_REPLICA 从隔离副本恢复，结果为一致且修复动作数为 1（5/5）；无故障时两种来源均保持一致，CID 不一致与同时缺失时均 Fail-Closed。恢复端到端时延中位数约 3.1~3.2 s。

表 7 E5 恢复结果与时长汇总

故障	对象来源	有效(n)	恢复判定	修复动作	时长中位数/ms
对象损坏	LOCAL_ONLY	5	FAIL_CLOSED	0	3112.2
对象损坏	KUBO_REPLICA	5	CONSISTENT	1	3129.6
CID 不一致	LOCAL_ONLY	5	FAIL_CLOSED	0	-
CID 不一致	KUBO_REPLICA	5	FAIL_CLOSED	0	-
同时缺失	LOCAL_ONLY	5	FAIL_CLOSED	0	-
同时缺失	KUBO_REPLICA	5	FAIL_CLOSED	0	-

综上，研究内容三目前已完成密文对象结构、版本语义、前瞻性撤销、材料释放判定、数据库任务状态机、链下对象存储与恢复协调，并完成 145 个有效运行的正式实验：状态一致性与幂等性全部通过，未观察到错误材料释放，撤销窗口保持 Fail-Closed。阶段性实验同时表明：HEADER_ONLY 的规模因素影响较小，BODY_ROTATION 在大 Body 规模下存在可观察的额外成本；Kubo 副本的核心作用是在特定本地对象损坏场景下提供可验证的恢复来源，正常路径未观察到稳定的性能优势，因此被定位为故障恢复可用性机制而非性能增强手段。全部结论以冻结配置、预注册统计与单节点环境为边界，不评估多 Validator 共识性能，不主张追溯撤销。

（7）三项研究内容的接口关系与整体系统闭环

三项研究内容通过统一的时间语义与策略摘要衔接，构成一条完整主线。研究内容一输出的唯一语义表示与策略摘要（policyDigest）进入研究内容二的资源注册、能力绑定与验证复核：资源状态记录 policyDigest，能力签发与验证都绑定该摘要，验证流程重新执行时间策略检查，从而把策略语义、策略摘要与链上状态绑定。研究内容二输出的可验证授权状态（资源状态、epoch、stateVersion、userVersion）成为研究内容三判断材料释放、Header 更新

与撤销闭环的可信输入：材料释放由 `AccessMaterialReleaseGuard` 依据链上复合状态判定，撤销事件驱动 `Header` 更新意图，链下对象版本与恢复动作与链上状态保持一致。反过来，研究内容三的每一次状态变化（撤销、`Header` 闭合、密钥轮换）都通过版本递增使旧请求与旧能力失效，约束研究内容二后续请求的有效性。

从系统运行角度看，一次完整的数据共享生命周期贯穿三项研究内容：策略创建与规范化生成唯一语义与摘要，资源与摘要注册到链上状态；数据用户获得由链上状态约束的能力并完成验证与一次性消费；材料释放组件依据链上复合状态释放密文材料；授权状态变化触发 `Header` 更新与材料释放判定切换；对象损坏通过恢复协调器从隔离副本验证并恢复。整个生命周期中，策略摘要、授权状态、任务状态与对象摘要相互绑定，任何一环的状态变化都会沿接口传导，形成“策略生成—状态锚定—材料释放—版本更新—撤销恢复”的反馈闭环。正是这种接口关系使三项研究内容构成有机整体，而不是三项独立工作的拼凑。

接口一致性是系统设计中被明确验证的属性：策略摘要由 `I*` 确定性计算，资源注册、能力签发、验证复核与材料释放判定均引用同一摘要，任何环节的摘要不一致都会导致流程拒绝；授权状态版本由链上合约单调维护，能力绑定与验证复核均以验证时点的确认快照为准，状态变化沿版本字段传导；`Header` 版本与对象摘要由链上注册表登记，材料释放判定同时校验链上状态与对象完整性，恢复路径以 `SHA-256` 摘要为完整性权威。三个接口分别对应“语义一致”“状态一致”“对象一致”，共同保证系统在正常路径与故障路径下都不会出现链上状态与链下执行脱节的情况。

整体完成度可汇总如下：三项研究内容均覆盖“模型—算法—实现—测试—实验”的完整链条，每项内容都有明确的阶段性结论、负结果与适用边界，为学位论文的后续写作提供了完整的证据基础。

表 8 三项研究内容进展总览

研究内容	当前状态	主要证据
研究内容一（策略确定性表示与编译）	核心方法与正式实验已完成	形式化模型、算法 1/2、81 项测试、98.61% 覆盖率、168 样本、15120 条有效记录；后续为论文级凝练与适用边界表述
研究内容二（许可链可信授权执行）	原型与正式实验已完成	五节点链、 <code>AuthorizationState</code> 、 <code>CAP2</code> 、共享 <code>Nonce</code> 、9720 运行块、链读取占比 98.66%~98.80%；后续为缓存与层次接口的协议级讨论
研究内容三（版本化密文头部与前瞻性撤销）	原型与正式实验已完成	版本协议、任务状态机、恢复协调、145 个有效运行、错误材料释放 0；后续为多节点扩展与批量更新压力
论文整体（全文整合与定稿）	章节草稿已形成	三项内容证据链与审计材料；后续为相关工作完善、理论深化、定稿

参考文献

[1] Saltzer J H, Schroeder M D. The protection of information in computer systems[J]. *Proceedings of the IEEE*, 1975, 63(9): 1278-1308.

- [2] Dennis J B, Van Horn E C. Programming semantics for multiprogrammed computations[J]. Communications of the ACM, 1966, 9(3): 143-155.
- [3] Miller M S, Yee K, Shapiro J. Capability myths demolished[R]. Technical Report SRL2003-02, Johns Hopkins University, 2003.
- [4] Lampson B W. Protection[J]. ACM SIGOPS Operating Systems Review, 1974, 8(1): 18-24.
- [5] Nakamoto S. Bitcoin: A peer-to-peer electronic cash system[EB/OL]. 2008. <https://bitcoin.org/bitcoin.pdf>.
- [6] Wood G. Ethereum: A secure decentralised generalised transaction ledger[EB/OL]. Ethereum Project Yellow Paper, 2014. <https://ethereum.github.io/yellowpaper/paper.pdf>.
- [7] Androulaki E, Barger A, Bortnikov V, et al. Hyperledger Fabric: A distributed operating system for permissioned blockchains[C]//Proceedings of the 13th EuroSys Conference. ACM, 2018: 30.
- [8] Bertino E, Bonatti P A, Ferrari E. TRBAC: A temporal role-based access control model[J]. ACM Transactions on Information and System Security, 2001, 4(3): 191-233.
- [9] Sandhu R S, Coyne E J, Feinstein H L, et al. Role-based access control models[J]. IEEE Computer, 1996, 29(2): 38-47.
- [10] Bethencourt J, Sahai A, Waters B. Ciphertext-policy attribute-based encryption[C]//Proceedings of the 2007 IEEE Symposium on Security and Privacy. IEEE, 2007: 321-334.
- [11] Sahai A, Waters B. Fuzzy identity-based encryption[C]//EUROCRYPT 2005. Springer, 2005: 457-473.
- [12] Goyal V, Pandey O, Sahai A, et al. Attribute-based encryption for fine-grained access control of encrypted data[C]//Proceedings of the 13th ACM Conference on Computer and Communications Security. ACM, 2006: 89-98.
- [13] Hardt D. The OAuth 2.0 authorization framework[S]. RFC 6749, 2012.
- [14] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT)[S]. RFC 7519, 2015.
- [15] Rouhani S, Belchior R, Cruz R S, et al. Distributed attribute-based access control system using permissioned blockchain[J]. World Wide Web, 2021, 24(5): 1617-1644.
- [16] Maesa D D F, Mori P, Ricci L. Blockchain based access control[C]//IFIP International Conference on Distributed Applications and Interoperable Systems. Springer, 2017: 206-220.
- [17] Boldyreva A, Goyal V, Kumar V. Identity-based encryption with efficient revocation[C]//Proceedings of the 15th ACM Conference on Computer and Communications Security. ACM, 2008: 417-426.
- [18] Abiteboul S, Manolescu I, Polyzotis N, et al. XML processing in DHT networks[C]//Proceedings of the 24th International Conference on Data Engineering (ICDE). IEEE,

2008: 606-615.

[19] Rundgren A, Jordan B, Erdtman S. JSON canonicalization scheme (JCS)[S]. RFC 8785, 2020.

[20] Claessen K, Hughes J. QuickCheck: A lightweight tool for random testing of Haskell programs[C]//Proceedings of ICFP 2000. ACM, 2000: 268-279.

[21] Hyperledger Besu Documentation. QBFT consensus protocol[EB/OL]. [2026-08-02]. <https://besu.hyperledger.org/private-networks/how-to/configure/consensus/qbft>.

[22] Josefsson S, Liusvaara I. Edwards-Curve digital signature algorithm (EdDSA)[S]. RFC 8032, 2017.

[23] Lamport L. Time, clocks, and the ordering of events in a distributed system[J]. Communications of the ACM, 1978, 21(7): 558-565.

[24] PostgreSQL Global Development Group. PostgreSQL 16 documentation: INSERT[EB/OL]. [2026-08-02]. <https://www.postgresql.org/docs/16/sql-insert.html>.

[25] Gray J. The transaction concept: Virtues and limitations[C]//Proceedings of the 7th International Conference on Very Large Data Bases. 1981: 144-154.

[26] Efron B. Bootstrap methods: Another look at the jackknife[J]. The Annals of Statistics, 1979, 7(1): 1-26.

[27] Dworkin M. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC[S]. NIST Special Publication 800-38D, 2007.

[28] Barnes R, Bhargavan K, Lipp B, et al. Hybrid public key encryption[S]. RFC 9180, 2022.

[29] Benet J. IPFS - Content addressed, versioned, P2P file system[EB/OL]. arXiv:1407.3561, 2014[2026-08-02]. <https://arxiv.org/abs/1407.3561>.

参考文献

[1] Saltzer J H, Schroeder M D. The protection of information in computer systems[J]. Proceedings of the IEEE, 1975, 63(9): 1278-1308.

[2] Dennis J B, Van Horn E C. Programming semantics for multiprogrammed computations[J]. Communications of the ACM, 1966, 9(3): 143-155.

[3] Miller M S, Yee K, Shapiro J. Capability myths demolished[R]. Technical Report SRL2003-02, Johns Hopkins University, 2003.

[4] Lampson B W. Protection[J]. ACM SIGOPS Operating Systems Review, 1974, 8(1): 18-24.

[5] Nakamoto S. Bitcoin: A peer-to-peer electronic cash system[EB/OL]. 2008. <https://bitcoin.org/bitcoin.pdf>.

[6] Wood G. Ethereum: A secure decentralised generalised transaction ledger[EB/OL]. Ethereum Project Yellow Paper, 2014. <https://ethereum.github.io/yellowpaper/paper.pdf>.

[7] Androulaki E, Barger A, Bortnikov V, et al. Hyperledger Fabric: A distributed operating system for permissioned blockchains[C]//Proceedings of the 13th EuroSys Conference. ACM, 2018: 30.

- [8] Bertino E, Bonatti P A, Ferrari E. TRBAC: A temporal role-based access control model[J]. ACM Transactions on Information and System Security, 2001, 4(3): 191-233.
- [9] Sandhu R S, Coyne E J, Feinstein H L, et al. Role-based access control models[J]. IEEE Computer, 1996, 29(2): 38-47.
- [10] Bethencourt J, Sahai A, Waters B. Ciphertext-policy attribute-based encryption[C]//Proceedings of the 2007 IEEE Symposium on Security and Privacy. IEEE, 2007: 321-334.
- [11] Sahai A, Waters B. Fuzzy identity-based encryption[C]//EUROCRYPT 2005. Springer, 2005: 457-473.
- [12] Goyal V, Pandey O, Sahai A, et al. Attribute-based encryption for fine-grained access control of encrypted data[C]//Proceedings of the 13th ACM Conference on Computer and Communications Security. ACM, 2006: 89-98.
- [13] Hardt D. The OAuth 2.0 authorization framework[S]. RFC 6749, 2012.
- [14] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT)[S]. RFC 7519, 2015.
- [15] Rouhani S, Belchior R, Cruz R S, et al. Distributed attribute-based access control system using permissioned blockchain[J]. World Wide Web, 2021, 24(5): 1617-1644.
- [16] Maesa D D F, Mori P, Ricci L. Blockchain based access control[C]//IFIP International Conference on Distributed Applications and Interoperable Systems. Springer, 2017: 206-220.
- [17] Boldyreva A, Goyal V, Kumar V. Identity-based encryption with efficient revocation[C]//Proceedings of the 15th ACM Conference on Computer and Communications Security. ACM, 2008: 417-426.
- [18] Abiteboul S, Manolescu I, Polyzotis N, et al. XML processing in DHT networks[C]//Proceedings of the 24th International Conference on Data Engineering (ICDE). IEEE, 2008: 606-615.
- [19] Rundgren A, Jordan B, Erdtman S. JSON canonicalization scheme (JCS)[S]. RFC 8785, 2020.
- [20] Claessen K, Hughes J. QuickCheck: A lightweight tool for random testing of Haskell programs[C]//Proceedings of ICFP 2000. ACM, 2000: 268-279.
- [21] Hyperledger Besu Documentation. QBFT consensus protocol[EB/OL]. [2026-08-02]. <https://besu.hyperledger.org/private-networks/how-to/configure/consensus/qbft>.
- [22] Josefsson S, Liusvaara I. Edwards-Curve digital signature algorithm (EdDSA)[S]. RFC 8032, 2017.
- [23] Lamport L. Time, clocks, and the ordering of events in a distributed system[J]. Communications of the ACM, 1978, 21(7): 558-565.
- [24] PostgreSQL Global Development Group. PostgreSQL 16 documentation: INSERT[EB/OL]. [2026-08-02]. <https://www.postgresql.org/docs/16/sql-insert.html>.
- [25] Gray J. The transaction concept: Virtues and limitations[C]//Proceedings of the 7th International Conference on Very Large Data Bases. 1981: 144-154.
- [26] Efron B. Bootstrap methods: Another look at the jackknife[J]. The Annals of Statistics, 1979, 7(1): 1-26.
- [27] Dworkin M. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC[S]. NIST Special Publication 800-38D, 2007.

[28] Barnes R, Bhargavan K, Lipp B, et al. Hybrid public key encryption[S]. RFC 9180, 2022.

[29] Benet J. IPFS - Content addressed, versioned, P2P file system[EB/OL]. arXiv:1407.3561, 2014[2026-08-02].
<https://arxiv.org/abs/1407.3561>.

4. 阶段性研究成果

[1] 王威, 夏琦, 高建彬, 夏虎. 基于许可联盟链状态锚定与共享 **Nonce** 的授权执行方法[J]. 软件学报 (论文初稿已完成, 拟投稿) .

[2] 王威, 高建彬, 王鹏. 一种非连续时间访问策略的确定性编译方法及系统: 中国, [P]. (专利撰写中, 拟申请) .

[3] 王威, 高建彬, 王鹏. 一种链上可信授权与版本化密文对象管理方法及系统: 中国, [P]. (专利撰写中, 拟申请) .

另: 三套可复现原型与冻结实验数据集 (时间策略编译、许可链授权执行、版本化密文头部与撤销恢复)。

二、存在的主要问题和解决办法

1. 未按开题计划完成的研究工作，研究工作存在的原理性、技术性难题以及在实验条件等方面的限制（可续页）

总体来看，当前研究已形成“策略表示—可信授权执行—密文生命周期治理”的技术闭环，三项研究内容的核心机制与正式实验均已达到阶段性完成状态。对照中期考评对“技术路线基本闭合、后续转向整合与深化”的定位，当前仍存在以下三个需要进一步推进的问题。

（1）三项研究内容的整体理论贯通与统一抽象仍需加强。研究内容一至三分别形成了较为完整的方法、原型与实验结果，但作为一篇完整的中期研究报告，需要把时间语义表示、链上状态锚定、密文对象版本化三者的接口关系与安全假设整理为连续、自洽的学术论证；研究内容二的安全属性与故障模型、研究内容三的版本一致性论证目前仍以实验验证为主，需要进一步明确实验验证与形式化证明的边界，避免对安全性质作过强概括。

（2）关键机制的对比论证与实验支撑仍需强化。本报告引用的相关路线（普通令牌、属性基加密、中心化授权、去中心化存储等）在背景部分已有初步对照，但面向时间约束策略表示、状态锚定执行、版本化撤销传导等机制的定量对比仍不充分；实验方面，缓存与层次覆盖的协议级收益、批量密文头更新与更多验证实例的一致性表现尚未在更大规模下验证，现有结论的适用范围需要随证据边界如实表述。

（3）最终论文集成、规范表达与成果凝练仍需推进。三项研究内容的核心机制与正式实验均已形成，但作为中期报告后续向学位论文过渡，仍需统一符号与术语体系、完善相关工作综述、规范参考文献与图表的交叉引用，并把阶段性成果整理为符合发表规范的论文与专利文本。从计划管理角度看，上述问题的解决顺序为：先完成论文结构整合与理论表述（问题一），再完善相关工作与创新边界（问题三），随后在条件允许时补充扩展实验与对比实验（问题二、问题四），最后完成全文定稿。这一顺序保证论文主线始终清晰，扩展实验服务于结论而不占用主线时间；每项问题都有明确的解决措施、验证方式与完成标准，进度可在后续阶段对照检查，避免出现“问题列出但无跟进”的情况。

在结论表述方面，现有负结果（缓存与层次覆盖无稳定端到端收益、Kubo 正常路径无稳定性能优势）均基于配对比较与 95% 置信区间跨越 0 的统计判断，而非单一运行的平均值比较；论文写作需要保留这一统计边界，避免把“未观察到稳定差异”表述为“两者等价”或“证明无差异”。统计推断以冻结的重复次数与预注册规则为边界，任何超出该边界的推断都需要补充实验支持。

2. 针对上述问题采取何种解决办法，对学位论文的研究内容及所采取的理论方法、技术路线和实施方案的进一步调整，以及下一步研究计划（可续页）

针对上述问题，后续将沿理论贯通、对比强化、集成凝练三条主线继续推进，每条问题均有对应的解决办法与可验证的完成标准，具体时间安排统一在后续研究计划中说明。

（1）针对理论贯通与统一抽象问题，将按“统一语义—可信授权执行—密文生命周期闭合”的主线重构论证结构，统一符号与术语，把三项研究内容之间的接口关系整理为可引用的命题形式；理论层面补充研究内容二的安全属性说明与研究内容三的版本一致性论证，明确区分实验验证与形式化证明的边界。验证方式为逐章对照审稿检查表，完成标准为各研究内容接口一致、术语统一、边界表述与冻结证据一致。

（2）针对对比论证与实验支撑问题，将基于已核验的文献扩充相关工作综述，以对比表整理各路线在时间语义表达、状态锚定、重放控制、撤销传导与恢复能力上的差异；对每一处创新性表述建立证据对照。实验方面按预注册设计补充协议级收益与规模扩展实验，每项扩展先明确假设与指标再执行，无法完成的扩展如实列入论文局限与未来工作。验证方式为文献来源抽查与实验设计预注册检查，完成标准为创新边界与证据一致、扩展实验边界明确。

（3）针对论文集成与成果凝练问题，将统一全文符号与参考文献系统，完善图表交叉引用与阶段性成果的规范表述；论文初稿围绕研究内容二或三组织核心章节，专利文本按当前方案方向撰写，所有成果状态均采用“已完成初稿/拟投稿/拟申请”的稳妥表述，不虚构发表或授权状态。验证方式为格式规范逐项检查，完成标准为中期报告可直接作为学位论文相关章节的写作基础。下一步具体研究计划（时间以实际中期考评与毕业安排为准）：

阶段 1（2026 年 9 月—10 月）：完成三项研究内容的论文级整合，统一术语、符号与接口表述；完善相关工作综述与既有路线对比，形成创新边界说明；完成论文整体结构与章节篇幅规划。

阶段 2（2026 年 11 月—12 月）：深化理论表述与结论边界整理，按需补充针对性扩展实验（缓存协议收益、批量 Header 更新、更多 Verifier 一致性等）；完成全文图表、公式与参考文献规范化，以及一致性、数字与引用审计。

阶段 3（2027 年 1 月—3 月）：完成学位论文全文定稿、格式审查与盲审准备；根据导师与专家意见进行最后一轮修改，并同步完成答辩材料准备。

各阶段均以可交付成果为节点：阶段 1 交付论文结构与章节草稿、相关工作综述与创新边界说明；阶段 2 交付理论表述完善稿、扩展实验报告与规范化后的全文；阶段 3 交付提交盲审的完整论文。若某阶段因客观原因延迟，将优先保证论文主线质量，再补充扩展内容，并通过阶段检查及时调整安排。

上述计划的时间节点以实际中期考评时间与学校毕业安排为准，若考评时间或培养计划调整，各阶段起止时间相应平移，但阶段顺序与交付成果保持不变。计划的可行性建立在已有工作基础上：三项研究内容的核心实验均已冻结，后续以写作、整合与针对性验证为主，

风险可控、可跟踪。

三、中期考评审查意见

1. 导师对工作进展及研究计划的意见:

年 月 日

年 月 日

2. 中期考评专家组意见

考评日期		考评地点	
考评专家			
考评成绩	合格____票	基本合格____票	不合格____票
结 论	☐ 通过 ☐ 原则通过 ☐ 不通过 通过： 表决票均为合格 原则通过： 表决票中有 1 票为基本合格或不合格，其余为合格和基本合格 不通过： 表决票中有 2 票及以上为不合格		

对学位论文工作进展以及下一步研究计划的建议, 是否适合继续攻读学位:

年 月 日

3. 学院意见:

年 月 日